Meet your code detective.

An illustrated guide for a Class 5 reader. You do not need to know C yet.

CodeSentinel helps you look inside a small program.

See its pieces. Follow its choices. Find things to check.

We will learn with a tiny sum and simple pictures.



Our detective reads the code. It does not run the C program.

Code is like a recipe.

A recipe is a list of instructions. Code is a list of instructions too.

A RECIPE

1. Take two apples.
2. Cut them.
3. Put them in a bowl.

A LITTLE PROGRAM

1. Work out a sum.
2. Keep its answer.
3. Give the answer back.

CodeSentinel is like someone reading the recipe and explaining it. Reading a recipe is different from cooking the meal.

The coding word for the written recipe is source code.

Take your first three steps.

Ask an adult or teammate to start the app. Then open the easy guide.

1

Start here

Choose “New to coding? Start here”.

2

Tiny example

Choose “Load tiny example”. It replaces the editor contents.

3

Analyze

Choose “Analyze this code”. Then try “Next idea”.

You can choose a lesson directly, go Back, or open the real result when you feel ready.

Here is our little program.

This is the actual Arithmetic sample used in the app.

```
int calculate(void) {
    int a = 2 * 8 + 1;
    int b = a + 3;
    return b;
}
```

calculate is the name of a little job.

a and **b** are names that help us keep values.

return gives a value back.

The * sign means multiply.

We will read the math together: 2 × 8 = 16, then 16 + 1 = 17, then 17 + 3 = 20.

This explanation uses math. The tool does not run the C program.

First, find the little pieces.

A sentence has words and punctuation. Code has pieces too.

```
int a = 2 * 8 + 1;
```

int
whole-number type

a
name

=
assign a value

2
number

*****
multiply

8
number

+
add

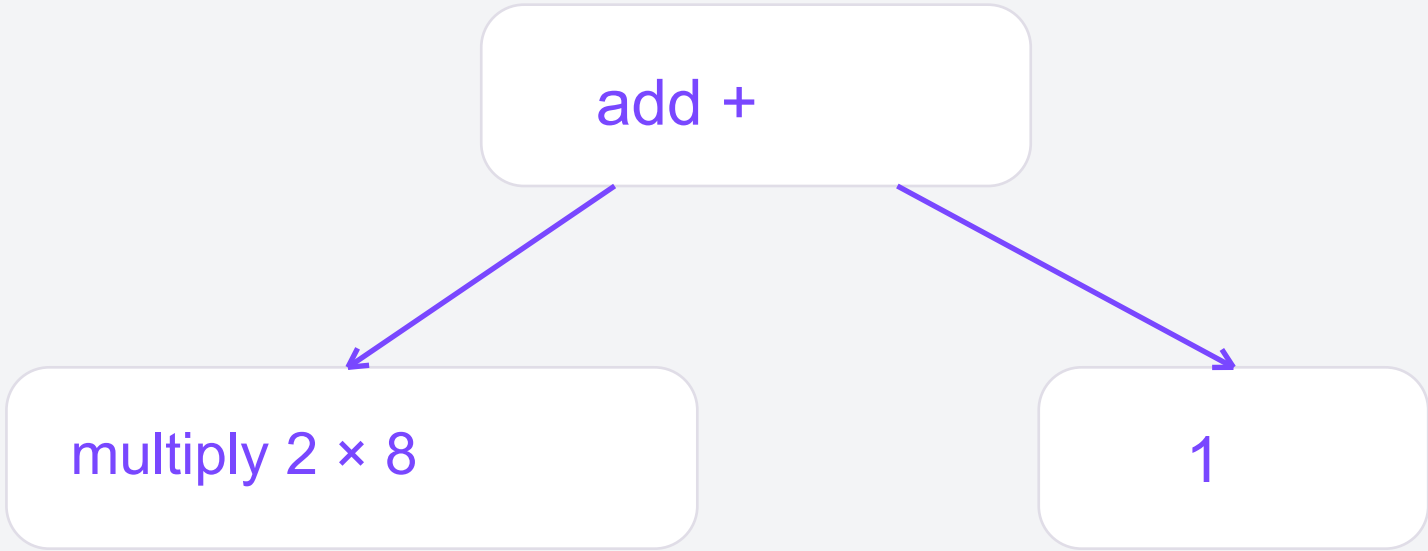
1
number

;
end of instruction

The coding word for these pieces is tokens.

Then, build a tree of parts.

A tree shows which pieces belong together. This is a teaching picture.



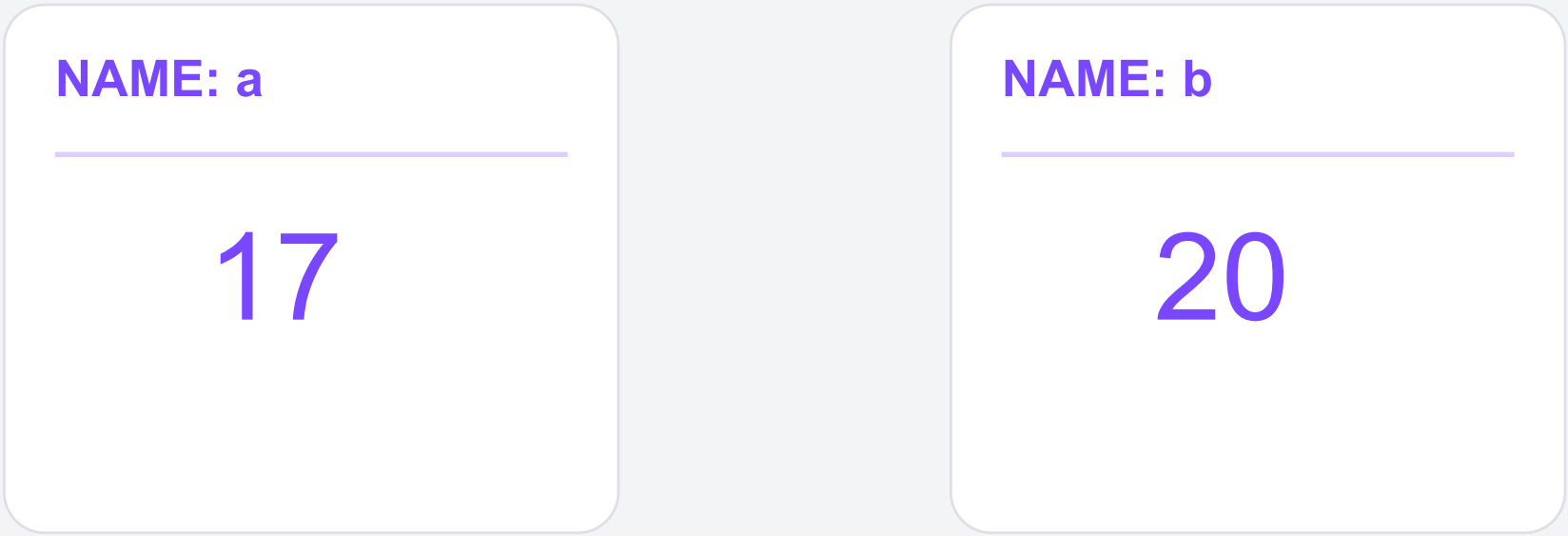
The multiply part belongs inside the add part.

Work out 2 × 8 first. Then add 1.

The code tree is called an AST: abstract syntax tree.

Keep values in named boxes.

This picture helps us imagine variables. It is not a drawing of real memory.



a holds the answer to 2 * 8 + 1.
b holds the answer to a + 3.

A variable gives a place for a value a name.

Same name, different place.

Imagine two classrooms with a box called pencils. This is an analogy.

CLASSROOM A

Box name: pencils
The label belongs to Classroom A.

CLASSROOM B

Box name: pencils
The label belongs to Classroom B.

The name alone is not enough. We also need its place.

Code can have the same name in different scopes. The tool gives each declaration its own identity.

A symbol table is the list of names and the places they belong.

Turn a big sum into small steps.

Small steps make it easier to see where an answer came from.

1 $2 \times 8 = 16$

2 $16 + 1 = 17$

3 $17 + 3 = 20$

The real tool uses temporary values for small calculations. Its small-step list is called TAC: three-address code.

Make supported calculations simpler.

Two different ideas help explain the Arithmetic result.

FOLDING

Work out a constant sum.

2 * 8 becomes 16.

PROPAGATION

Carry a known value forward.

If a is 17, use 17 in a + 3.

The final return in the optimized Arithmetic result becomes **return 20**.

Some calculations cannot be safely simplified. The tool must explain its limits.

A simpler calculation is not a measured speedup.

Choices make different roads.

This pretend umbrella story explains a map of choices.

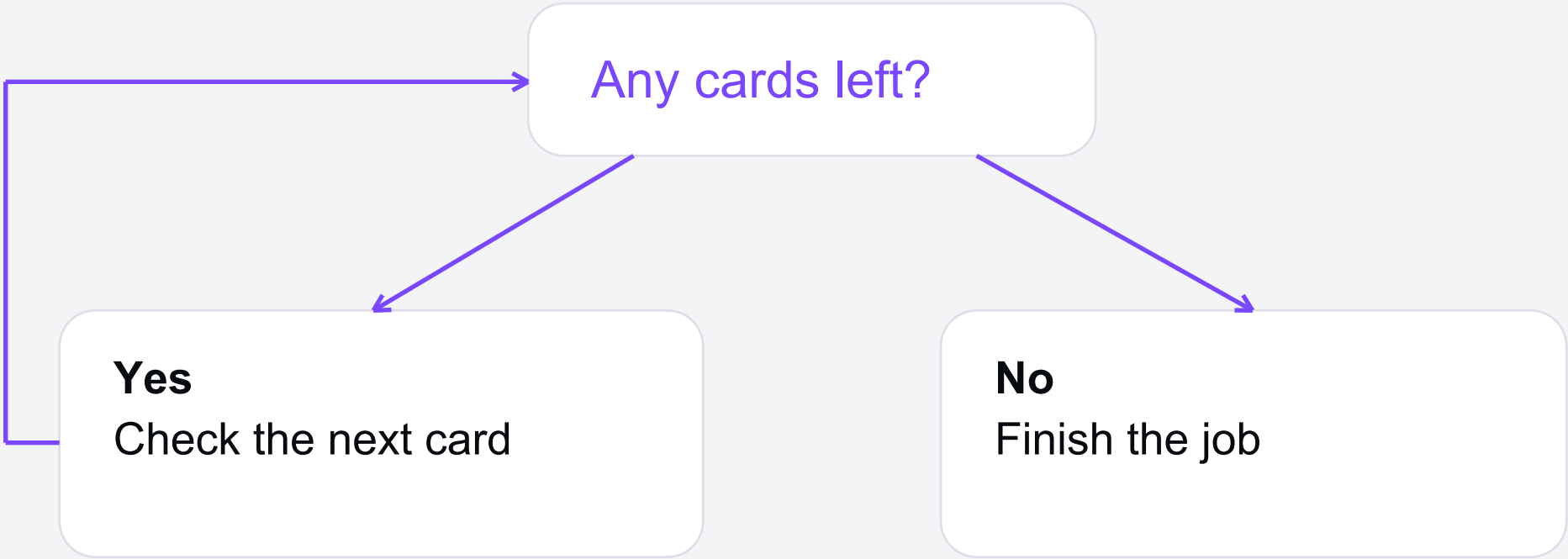


A program can have choices too. A CFG, or control-flow graph, shows possible roads between groups of steps.

A road map shows possibilities. It is not a recording of a program running.

A loop lets a road go around again.

Imagine checking three cards, one at a time. This is a teaching story.



After one card, ask again. The arrow back is the loop.
In code, continue goes to the loop's next step; break leaves the loop.

Follow the values along the roads.

We can ask two questions without running a program.

LOOK AHEAD

Will a later step need this value?

Coding words: live variables.

LOOK BEHIND

Which earlier write might have supplied this value?

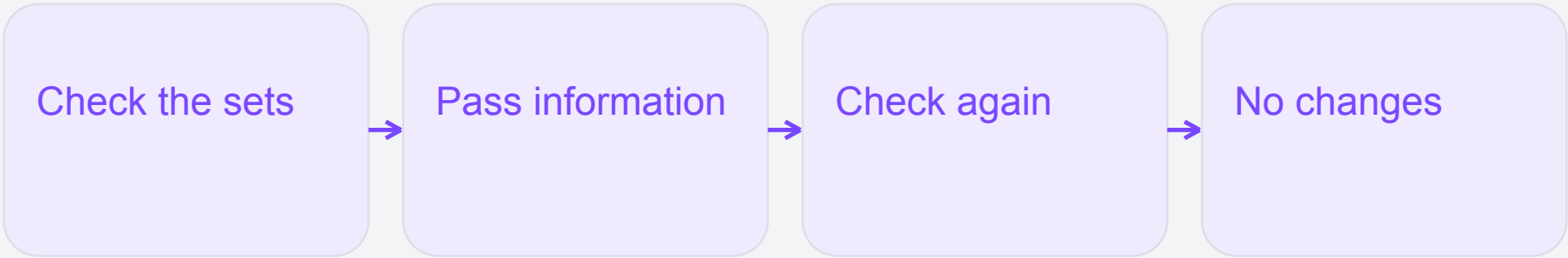
Coding words: reaching definitions.

If one road writes x = 1 and another writes x = 2, a later read may receive either write. Keep both possibilities.

This tool follows named values. It does not know all memory contents.

Check again until the sets settle.

Sets are groups of facts. A fixed point means another check changes nothing.



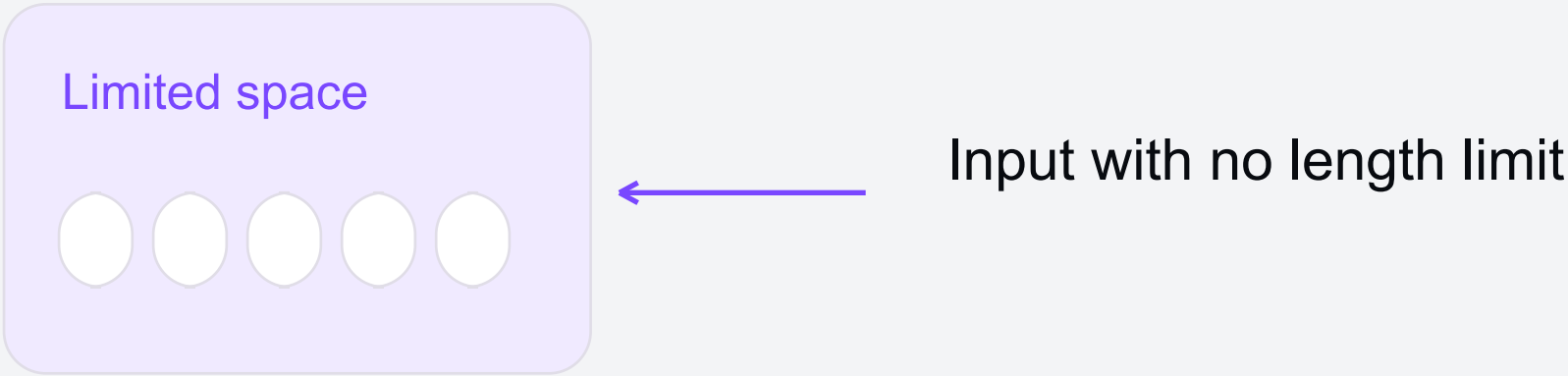
A loop can make information travel around the map.

The tool repeats its set calculations. When the next round changes nothing, the sets have settled.

These rounds are analysis checks. They are not runs of your C program.

A warning asks for a closer look.

Think of a teacher pointing out something to review.



In the Full pipeline sample, one input call has no length limit. Another copy call has no capacity argument. The tool points to their lines and explains why to review them.

A warning is not proof that an overflow or attack happens.

No findings means no findings in the checks that ran. It does not prove safety.

What if the tool has no answer?

Missing information must not pretend to be a correct result.

CODE CHANGED

The old answer belongs to the old recipe. Analyze again.

REQUEST FAILED

Read the message and retry. The easy guide hides old evidence.

UNSUPPORTED PART

Some C features are outside the tool's coverage. Read its reason.

NO RESULT

Unavailable is different from zero problems.

Our detective explains what it understood and what it could not check.

Try it in the real workbench.

This is an actual capture of the new Start here guide, using the real backend.

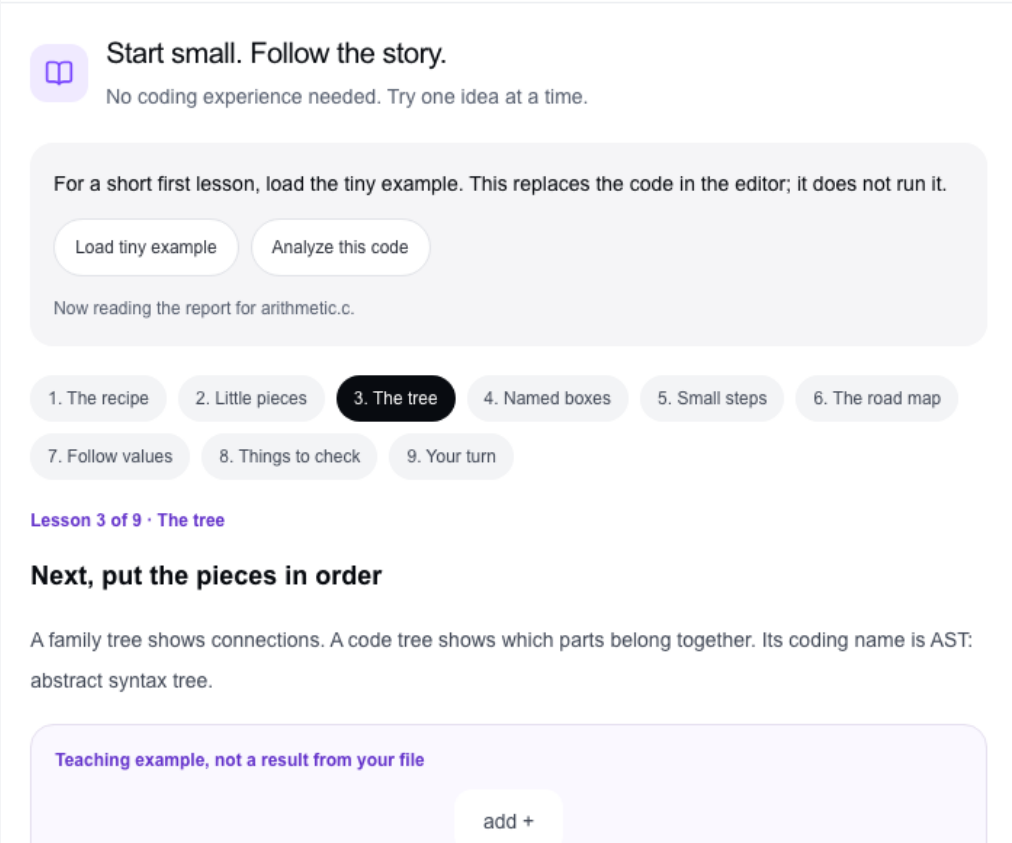
Choose **Start here**.

Load the tiny example.

Analyze it.

Follow each idea.

Use a source button to find the line behind a result.



The diagram inside the app is labeled as a teaching example. Detailed tabs keep the actual compiler outputs.

Now tell a friend the story.

You can answer these questions in your own words.

1. What is code like?
A recipe: a set of instructions.

2. Does CodeSentinel run the C program?
No. It reads the code and checks supported parts.

3. What can I see?
Pieces, names, small steps, possible roads and things to review.

4. Does a warning prove danger?
No. Look at its reason and its limits.

For grown-ups: based on the current local workbench, Arithmetic and Full pipeline teaching samples, and docs/BEGINNER_GUIDE.md. This is a plain-language introduction, not proof of a child's learning outcome. Local checks: production build and 21 real-backend Chromium cases. Some failure cases use controlled responses.